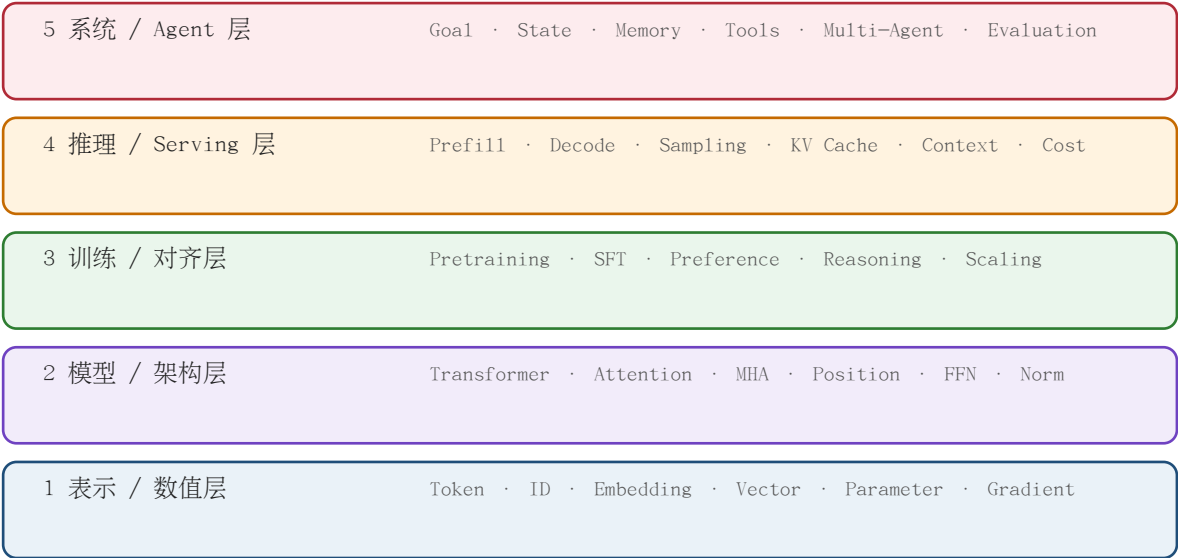


第30讲：从 LLM 基础课走向 Agent Systems

完整知识地图、科研接口与最低闭环能力

本讲目标
把第1-29讲从“课程目录”变成一张长期使用的心智地图。学完后你应该能：快速判断一篇论文在系统哪一层；把一个 Agent 现象拆成可观测变量；设计最小可复现实验；识别算力、通信、模型、检索和工具等混杂；并知道接下来学习 Agent Systems 时哪些基础已经够用、哪些暂时不必继续深挖。



以后读论文：先问“这篇论文改的是哪一层？测的是哪一层？混杂来自哪一层？”

这门课的真正终点不是“会背 Transformer”，而是：你已经能把一个大语言模型系统拆成表示、模型、训练、推理和 Agent/System 五个层级，并知道它们怎样相互影响。

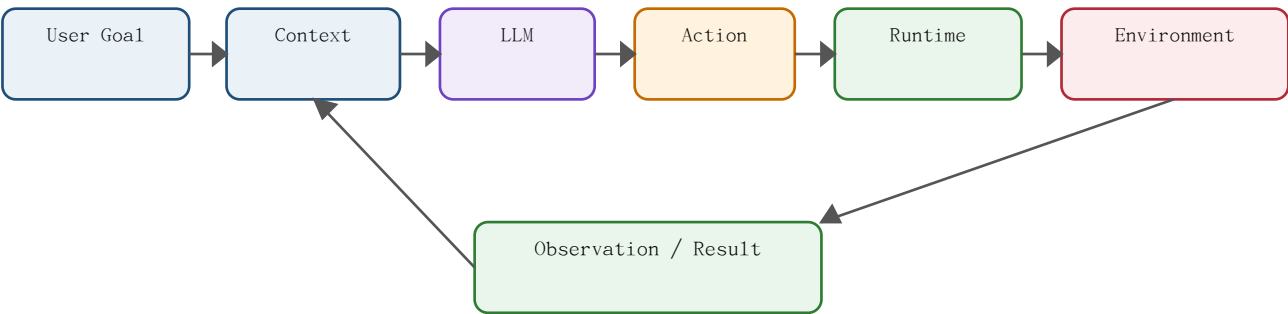
1. 第1-29讲到底学了什么？不是29个孤立知识点

可以把整门课压缩为五层。

层	你已经掌握的核心	典型问题
表示/数值层	Token、ID、Embedding、向量、参数、梯度	文字怎样进入神经网络？参数怎样被更新？
模型/架构层	Attention、QKV、MHA、Position、FFN、Norm、Decoder-only	LLM 内部怎样计算和传递上下文？
训练/对齐层	Pretraining、SFT、Preference、Reasoning、Scaling、Data	模型能力与行为是怎样形成的？
推理/Serving层	Sampling、Context、KV Cache、Prefill、Decode、Quantization	模型运行时为什么有随机性、延迟和成本？
System/Agent层	RAG、Memory、Tool、State、Planning、Multi-Agent	怎样把 LLM 变成持续决策系统？

以后读论文的第一个问题
先不要问“这篇论文厉不厉害”。先问：它主要改了哪一层？它声称改善的指标在哪一层？它有没有把别层的额外资源当成自己的方法收益？

2. 一条从文字到 Agent 行动的完整数据流



从 token 到系统：模型只是闭环中的一个组件。

把前29讲串起来，一个真实 Agent 请求大概会经历：

User Goal -> Chat Template / Context -> Tokenizer -> Token ID / Embedding -> Transformer Blocks -> Logits / Sampling -> Tool Call / Action -> Runtime Validation -> Environment -> Observation / Tool Result -> State / Memory Update -> Context -> LLM Inference

这条链很重要，因为任何一处都可能成为论文里的研究对象，也都可能成为失败来源。

3. 十个以后必须保持“条件反射”的核心概念

概念	看到它时立刻想到什么
Token	成本、上下文长度、输入输出单位

概念	看到它时立刻想到什么
Embedding	表示空间；RAG/Memory 检索接口
Attention	信息从哪些位置被聚合
Loss / Gradient	训练目标怎样变成参数变化
Context	本轮真正可见的信息，不等于长期记忆
KV Cache	当前推理计算缓存，不是经历
Grounding	回答是否被外部证据约束
State	系统当前 “认为世界是什么样”
Trajectory	Agent 真实运行的状态-动作-反馈路径
Error Correlation	多个 Agent 是否在同样地方一起错

4. 哪些数学和底层细节现在已经 “够用” ？

你现在不需要先补完一整套高等数学、概率论和优化理论才能做 Agent Systems。当前最低够用能力是：

你需要会	到什么程度
向量/矩阵	能看懂 shape、点积、线性变换、cosine
概率	理解分布、条件概率、sampling、平均值/方差
梯度	知道它决定参数更新方向，不要求手推复杂反传
指标	准确率、Recall、Precision、Correlation、Confidence Interval
实验	控制变量、随机种子、重复实验、消融、基线

暂时不要本末倒置
如果你的近期目标是 Agent Systems 论文，不必为了 “基础扎实” 先花半年推导所有 Transformer 公式。遇到论文确实需要的数学，再按问题补。

5. 从 LLM 基础进入 Agent Systems 后，研究对象发生了什么变化？

单个 LLM 主要研究“输入 -> 输出”；Agent Systems 更像研究一个随时间演化的动态系统：

```
State_t + Observation_t + Memory_t -> Policy -> Action_t -> Environment -> State_(t+1)
```

当有多个 Agent 时，还要加：

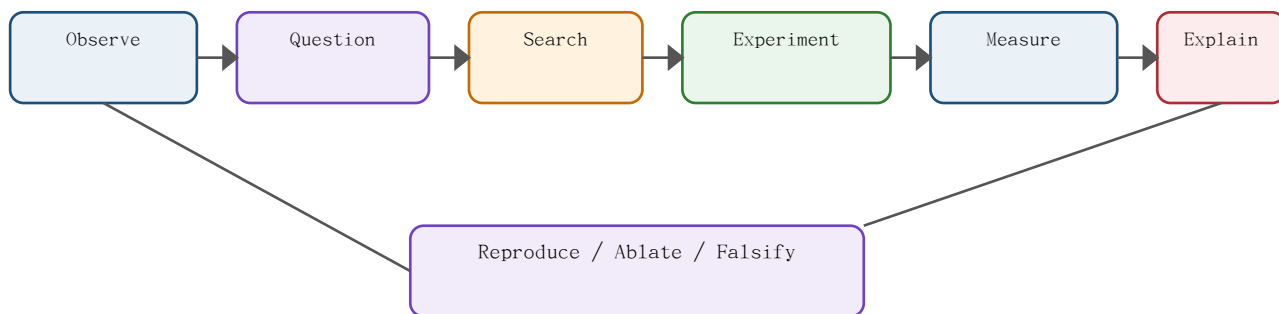
```
Communication + Shared/Private State + Coordination + Aggregation + Resource Allocation
```

核心转变
研究重点从“模型能不能答对一道题”扩展为“有限、随机、会犯错、会相互影响的 AI 组件组成的系统，怎样产生、传播、修复或放大行为”。

6. Agent Systems 的主要科研接口

接口	可研究现象	典型指标
Planning	计划长度、重规划、分解错误	Task Success、Steps、Recovery
Memory	写入、检索、过期、错误自强化	Recall、Memory Hit、Stale Rate
Tools	选错工具、参数错误、side effect	Selection/Argument/Execution Success
Communication	消息过多、锚定、信息增益	Messages、Tokens、Information Gain
Heterogeneity	角色/经历/工具差异	Behavior Divergence、Error Correlation
Coordination	分工、同步、冲突解决	Idle Time、Conflict、Task Completion
Evaluation	benchmark 与真实行为错位	Generalization、OOD、Cost
Reliability	错误传播、恢复、验证	Error Propagation、Recovery Rate
Collective Intelligence	集体收益与错误去相关	Collective Gain、Error Overlap

7. 把“现象”变成“研究问题”的标准流程



标准科研流程：先从真实现象出发，再进入假设和方法。

以后立项时建议始终遵循：

1) 现象/问题 2) 现象/问题 / 假设 / 方法 3) 现象/问题/假设/方法/实验 4) 现象/问题/假设/方法/实验/结果 5) 现象/问题/假设/方法/实验/结果/解释 6) 现象/问题/假设/方法/实验/结果/解释/验证 7) 现象/问题/假设/方法/实验/结果/解释/验证/结论

论文不是从方法开始

“我想加一个模块”不是好的科研起点。更好的起点是：我观察到了什么稳定现象？现有解释为什么不够？我能设计什么实验把不同解释区分开？

8. 研究问题 -> 可观测变量 -> 指标：一张接口表

研究问题	自变量/操纵	中间变量	结果指标
长期经历是否形成异质性	Experience 分配	Memory/Behavior Divergence	Error Correlation / Gain
通信是否真正有用	Message Policy / Topology	信息覆盖/一致性	Accuracy / Cost / Groupthink
多 Agent 是否优于单 Agent	Agent 数/交互机制	Calls/Tokens/Correlation	Compute-matched Gain
Memory 是否提高可靠性	Write/Retrieve Policy	Hit/Stale/False Memory	Success / Recovery
Tool 异质是否带来互补	Tool Set / Permission	Tool-use Divergence	Success / Side-effect / Cost
Verifier 是否能纠错	Verifier 类型/独立性	Accept/Reject Pattern	Error Reduction / False Reject

9. Agent 实验最常见的五个混杂

混杂	为什么危险	怎么控制
Compute	多 Agent 用更多 calls/tokens	compute-matched baseline
Model strength	不同组用了不同模型	固定模型或分层报告
Prompt length	方法组给了更多提示信息	匹配 context/token budget

混杂	为什么危险	怎么控制
Tool access	方法组工具更多	固定 Tool Set / 单独消融
Judge bias	同一 Judge 偏好某种风格	多 Judge / rule / human subset
Data leakage	测试题进入 prompt/memory/train	严格隔离与污染检查

最低闭环
如果一个实验没有基线、没有预算控制、没有运行轨迹、没有重复实验、没有失败分解，哪怕平均分高，也很难得到强机制结论。

10. 一篇 Agent Systems 论文应该怎样“拆读”？

先问	你要找什么
研究对象是什么？	单 Agent / Multi-Agent / Memory / Tool / Benchmark
现象是什么？	作者到底观察到了什么失败/规律
变量是什么？	哪些被操纵，哪些只是顺便变化
比较对象是谁？	baseline 是否公平、预算是否匹配
指标是什么？	只看 final score 还是有机制指标
代码/补充材料说什么？	Prompt、seed、模型、重试、异常处理
有没有异常现象？	表格、消融、失败案例里未解释的差异
局限是什么？	作者没覆盖的模型、任务、成本或边界

你以后找选题时，尤其应该盯住：表格里的异常差异、消融中的反直觉结果、不同模型/数据集之间不一致、失败案例、补充材料和 issue。

11. 你的“最低闭环能力”应该达到什么程度？

不要求你无 AI 手写完整框架，但至少应该能独立理解和验收下面这条实验链：

Define task -> Load benchmark -> Call model/API -> Save raw outputs/trajectories -> Compute metrics -> Repeat seeds -> Compare baselines -> Plot / Table -> Inspect failures -> Draw bounded conclusion	
能力	最低要求
Python	能改实验脚本、读 JSON/CSV、循环、函数、异常
API	知道 request/response、tokens、temperature、retry
Git	能保存实验版本与配置
Linux	能跑脚本、查日志、管理文件
Statistics	均值、标准差、置信区间、相关性
Paper reading	能定位方法、实验、限制、补充材料

12. 哪些内容现在可以“先不学深”？

暂时可浅学	原因
从零预训练大模型	资源不匹配近期 Agent 研究目标
CUDA kernel 优化	除非研究 serving / systems 性能
完整 RL 数学	只需按具体 Agent/RL 论文补
分布式训练细节	API/开源推理足以做很多低成本实验
所有 Agent 框架	机制理解比框架 API 更重要
复杂前端/产品部署	不是当前科研最低闭环

原则
先达到 “能提出问题 -> 能跑实验 -> 能解释结果 -> 能写论文” 的最小闭环，再按研究问题补深层知识。

13. 从基础课到 Agent Systems 的后续学习顺序

阶段	重点
A. 单 Agent 机制	Planning、Memory、Tool、Reflection、Evaluation
B. Multi-Agent 基础	Roles、Communication、Coordination、Aggregation
C. 系统可靠性	Failure Taxonomy、Recovery、Verification、Security
D. 集体智能	Diversity、Error Correlation、Specialization、Groupthink
E. 长期智能体	Persistent Memory、Experience、Adaptation、Identity
F. Scientific Agents	把上述机制放到科研任务中验证

14. 一个长期可用的 “研究问题过滤器”

问题	若答案是 “否” 意味着什么
现象真实且稳定吗？	先别设计方法
有科学/系统价值吗？	可能只是工程小技巧
别人是否已系统研究？	重新找边界或机制
能用低成本实验区分解释吗？	当前条件可能不适合
结果不刷 SOTA 也有价值吗？	避免论文完全押注性能
能在 3 个月形成闭环吗？	缩小问题规模
有可复现数据与指标吗？	先补 benchmark / measurement

15. 小实验：把 Agent 结果从“最终分数”拆成系统指标

材料包中的 Python 实验构造 12 个任务、3 个 Agent 的玩具结果，计算：

Individual Accuracy / Majority Accuracy / Collective Gain / Pairwise Error Correlation / Total Tokens / Calls

实验目的不是模拟真实模型，而是训练一种习惯：看到“多 Agent 更强”时，不要只看最终准确率，而要同时问错误依赖与资源预算。

科研习惯
一个表格最好同时包含 Outcome、Mechanism Metric 和 Cost。只有 Outcome，很难解释为什么；只有机制指标，又不知道是否真的有用。

16. 课程总复习：30讲知识地图

讲次	主题块
1-4	LLM、Token、Embedding、神经网络
5-7	训练、泛化、RNN/LSTM 到 Attention
8-12	Attention、MHA、位置、Block、GPT 架构
13-18	预训练、数据、Scaling、SFT、Preference、Reasoning
19-23	Inference、Sampling、Context、KV/Serving、Quantization/MoE
24-27	Hallucination、Memory、RAG、Tool Use
28-29	单 Agent、Multi-Agent
30	知识地图、研究接口、最低闭环

17. 你现在应该能解释的十个问题

- 1) 为什么 next-token prediction 能产生复杂能力，却又天然不等于事实验证？
- 2) Transformer 为什么依赖 Attention 和 Position？
- 3) Pretraining、SFT、Preference Learning 分别改变什么？
- 4) 为什么同一个模型同一个问题会给出不同答案？
- 5) Context、KV Cache、Parameter Memory、Agent Memory 有什么区别？
- 6) RAG 为什么能降低一部分幻觉，却不能保证正确？
- 7) Tool Call 为什么不等于 Tool 已经执行？
- 8) 一个 Agent 比普通 LLM 多了什么系统结构？
- 9) 多 Agent 为什么可能比单 Agent 更强，也可能只是更贵？
- 10) 为什么 Error Correlation 是集体智能研究中的核心变量？

如果这十个问题你能用自己的话解释，并能画出数据流图，那么这门基础课的目标已经基本达到。

18. 练习题

- 1) 把一个 LLM/Agent 论文分到“五层地图”中，为什么有助于发现实验混杂？
- 2) 为什么“方法组准确率更高”不能直接说明方法机制有效？
- 3) Agent Systems 中为什么 trajectory 是重要科研数据？
- 4) 如果研究 Memory，至少应该分别测哪些中间指标？
- 5) 为什么 compute-matched baseline 对 Multi-Agent 特别重要？
- 6) 如何区分 Answer Diversity 和 Functional Heterogeneity？
- 7) 为什么错误相关性可能比单体准确率更能解释集体收益？
- 8) 按照标准科研流程，观察到一个异常结果后下一步应该做什么？
- 9) 什么叫“最低闭环能力”？为什么它比先学完所有数学更重要？
- 10) 给自己设计一个 3 个月内可完成的 Agent 机制研究时，你会用哪些过滤条件？

答案见材料包《03_第30讲_练习题与答案.pdf》。

19. 最终总结

这门基础课真正建立的能力

你现在已经有了从“token 如何进入模型”一直追到“多个 Agent

如何形成系统行为”的完整链条。以后不管遇到新的模型、框架、Agent 产品或论文，都可以先把它放回这张地图：它改变哪一层？新增什么状态？怎样和环境交互？用什么指标证明？成本和混杂是什么？

接下来再学习 Agent Systems 时，就不需要重新从“Agent 是什么”开始，而是可以直接进入更像科研训练的课程：失败模式、评测、Memory、Tools、Communication、Coordination、Collective Intelligence、Persistent Agents，以及真正的论文实验设计。

第30讲结束：LLM 基础阶段完成。